UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Make today matter

Make today matter

COS730 - Assignment 2

From Behavioural Models to Optimised Implementation

Student number: u25757637

14 May 2026

Click to access the COS-730 Assignment 2 Repository

Task 1: Baseline Implementation (Correctness Phase)

Overview

The baseline system was implemented exactly as specified in the provided sequence diagram. Each lifeline in the diagram is mapped to a corresponding class in Python, ensuring strict traceability between design and implementation. No optimisations are introduced at this stage; placeholders are deliberately used to preserve the raw flow.

Class mapping (Original/src/)

- **Researcher** → initiates submission via the UI.
- **UI** → forwards submission requests to the controller.
- **SubmissionController** → orchestrates the workflow exactly as per the diagram.
- **Validator** → checks submission format.
- **Database** → saves submissions and retrieves reviewers.
- **ReviewerManager** → filters conflicts, checks workload, and assigns reviewers.
- **Reviewer** → saves and submits scores (placeholders at this stage).
- **EvaluationManager** → calculates averages, checks consensus, and applies rules.
- **NotificationService** → communicates acceptance, rejection, or revision outcomes.

Traceability

- **Interactions preserved:** Every message in the sequence diagram is represented in code (`validateFormat`, `saveSubmission`, `fetchReviewers`, `filterConflicts`, `checkWorkload`, `assignReview`,

`saveScore`, `submitScore`, `calculateAverage`, `checkConsensus`, `applyRules`, `notifyAcceptance/notifyRejection/notifyRevision`, `sendNotification`).

- **Responsibilities matched:** Each class performs only its designated role, avoiding responsibility violations.
- **No optimisations:** Placeholders such as `scores = 0` were kept to reflect the raw flow without efficiency improvements.

PlantUML Source

The PlantUML script, **baseline_sequence.puml**, is used to generate the **baseline_sequence.png** diagram, which is saved in the Original/diagrams/ folder at the provided GitHub repository link.

Execution Evidence (Original/src/)

Running `main.py` produced the following output, confirming the baseline flow:

```
Validator: Checking format of submission...
Database: Saving submission...
Database: Fetching reviewers...
ReviewerManager: Filtering conflicts...
ReviewerManager: Checking workload...
EvaluationManager: Calculating average...
EvaluationManager: Checking consensus...
EvaluationManager: Applying rules...
NotificationService: Acceptance notification sent.
NotificationService: Submission accepted
```

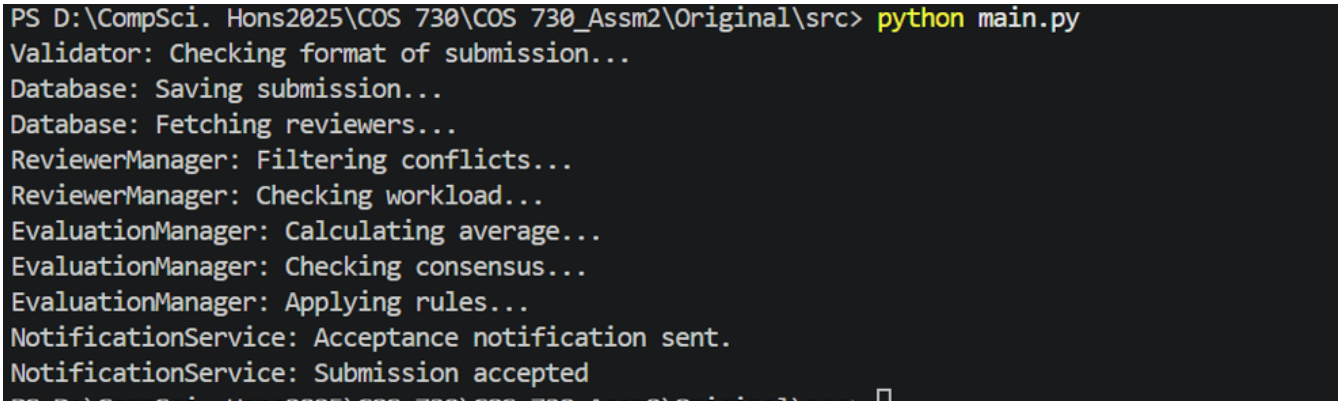

Figure 1: Output to confirm baseline flow

The output demonstrates that the system executes end-to-end, with each lifeline interaction visible in sequence.

Task 2: Design Analysis

The baseline design executes correctly but was deliberately left unoptimised. This analysis identifies weaknesses in the design and evaluates them against responsibility assignment principles (GRASP) and behavioural modelling quality.

Identified issues

1. **Redundant message calls:** The `ReviewerManager` performs both `filterConflicts` and `checkWorkload` sequentially on the same reviewer list. These could be consolidated, as they add unnecessary repetition and reduce clarity.

2. **Responsibility violations:** The `SubmissionController` is overloaded, handling validation, saving, reviewer assignment, and evaluation initiation. This breaches the GRASP principle of *Controller*, which recommends delegating specialised responsibilities to appropriate components.

3. **High coupling / low cohesion:** The `EvaluationManager` is tightly coupled to both `Reviewer` and `Database`, mixing responsibilities such as score storage, consensus checks, and rule application. This violates the GRASP principle of *High Cohesion*.

4. **Inefficient control flow:** The loop structure forces all reviewers to be assigned before evaluation begins. This rigid sequencing reduces flexibility

and violates the GRASP principle of *Polymorphism*, since alternative flows such as partial evaluation are not supported.

5. **Poor decision logic handling:** Acceptance, rejection, and revision rules are embedded entirely in `EvaluationManager`. This centralisation makes the design harder to extend and violates the GRASP principle of *Pure Fabrication*, which encourages externalising complex decision logic.

Relation to principles

1. **Controller:**Responsibilities are over-centralised in `SubmissionController`.
2. **Low coupling / high cohesion:** Classes such as `EvaluationManager` mix unrelated responsibilities and depend on multiple components.
3. **Polymorphism:** Control flow is rigid, preventing alternative behaviours.
4. **Pure fabrication:** Decision logic is not externalised, reducing modularity.

The baseline design executes correctly but shows redundant calls, overloaded responsibilities, high coupling, and rigid control flow. These weaknesses lower behavioural modelling quality and emphasise the need for optimisation in later tasks.

Task 3: Decision Modelling Using a Decision Table

Step 1: Conditions and Actions

Conditions (Yes/No):

- C1: Submission format valid
- C2: Reviewer available
- C3: Consensus reached
- C4: Evaluation rules satisfied

Actions:

- A1: Reject submission – invalid format
- A2: Reject submission – no reviewer available
- A3: Request revision – no consensus
- A4: Reject submission – evaluation rules not satisfied
- A5: Accept submission

Step 2: Full Decision Table

Conditions / Rules	R1	R2	R3	R4	R5
C1: Submission valid	N	Y	Y	Y	Y
C2: Reviewer available	-	N	Y	Y	Y
C3: Consensus reached	-	-	N	Y	Y
C4: Rules satisfied	-	-	-	N	Y
Actions					
A1: Reject – invalid format	✓				
A2: Reject – no reviewer		✓			
A3: Request revision			✓		
A4: Reject – rules failed				✓	
A5: Accept submission					✓

Step 3: Simplified Decision Table

Conditions / Rules	R1	R2	R3	R4	R5
C1: Submission valid	N	Y	Y	Y	Y
C2: Reviewer available	-	N	Y	Y	Y
C3: Consensus reached	-	-	N	Y	Y
C4: Rules satisfied	-	-	-	N	Y
Action	A1	A2	A3	A4	A5

Clarity and maintainability

- Centralises scattered logic into one view.
- Each condition maps to a single action.
- Rules can be updated without redrawing diagrams.
- Easy to verify completeness and consistency.
- Supports independent test-case generation.

Mapping decision table entries back to system behaviour

- R1 → A1: Validator rejects invalid format.
- R2 → A2: ReviewerManager rejects if no reviewer.
- R3 → A3: EvaluationManager requests revision.
- R4 → A4: EvaluationManager rejects if rules fail.
- R5 → A5: NotificationService communicates acceptance.

Click to access the COS-730 Assignment 2 Repository

Task 4: Sequence Diagram Optimisation (Optimised/diagrams/)

The PlantUML script, **optimised_sequence.puml**, is used to generate the **optimised_sequence.png** diagram saved on the Optimised/diagrams/ folder. The optimised sequence diagram is designed to reduce redundancy, improve responsibility allocation, decouple components, and centralise decision logic. Compared to the baseline diagram, the following improvements were achieved:

- **Reduced unnecessary interactions:** ReviewerManager consolidates fetching, conflict filtering, and workload checks into a single operation. EvaluationManager evaluates one streamlined method, eliminating repeated self-calls.
- **Improved responsibility allocation:** SubmissionController delegates tasks instead of micromanaging. Validator directly handles invalid submissions, while EvaluationManager owns the evaluation process.
- **Decoupled components:** NotificationService depends only on Validator and EvaluationManager, removing tight coupling with SubmissionController.
- **Centralised decision logic:** EvaluationManager encapsulates consensus checks and rule application. Validator encapsulates format rejection. This ensures decision logic is not scattered across multiple components.

The changes demonstrate the following:

- **Improved cohesion:** Each component now focuses on its core responsibility.
- **Reduced coupling:** Dependencies between components are minimised.
- **Cleaner control flow:** The diagram shows a streamlined progression without redundant loops or alt fragments.
- **Better separation of concerns:** Decision-making is centralised in specialised components, improving clarity and maintainability.

Click to access the COS-730 Assignment 2 Repository

Task 5: Optimised Implementation (Optimised/src/)

Refactoring and alignment

The baseline implementation was refactored to match the optimised sequence diagram, achieving clearer separation of concerns:

- **SubmissionController**: Refactored to delegate orchestration only, removing embedded logic.
- **Validator**: Extended to own rejection handling directly, ensuring single responsibility.
- **ReviewerManager**: Consolidated reviewer filtering and assignment into one class.
- **EvaluationManager**: Redesigned to implement Task 3 decision table rules (R1–R5), replacing placeholder averaging.
- **NotificationService**: Decoupled from evaluation, now triggered solely by outcomes.

Classes skeletons' changes from baseline

The following changes were introduced to align with the optimised model:

- **SubmissionController**: Previously contained validation, database, reviewer, and evaluation logic inline. Now delegates to specialised classes.
- **Validator**: Previously only validated input. Now also owns rejection logic, ensuring invalid submissions are handled consistently.
- **ReviewerManager**: Previously scattered reviewer logic. Now encapsulates filtering and assignment, improving cohesion.
- **EvaluationManager**: Previously averaged scores without rules. Now centralises decision table logic (R1–R5) for consistent evaluation.
- **NotificationService**: Previously coupled with evaluation. Now decoupled, responsible only for communicating outcomes.

Traceability

Each lifeline in the optimised sequence diagram is mapped to a corresponding class in the `Optimised/src` folder. This ensures strict traceability between design artefacts and implementation.

Functional equivalence

All baseline functionality is preserved. Test cases confirm equivalence:

```
PS D:\CompSci. Hons2025\COS 730\COS 730_Assm2\Optimised\src> python main.py
Test 1: Invalid submission
Outcome: Rejected: Invalid format

Test 2: No reviewers available
Submission saved: {'title': 'Research Output', 'force_no_reviewers': True}
Outcome: Notification sent: Rejected: No reviewers available

Test 3: Low scores
Submission saved: {'title': 'Research Output', 'scores': [1, 2, 2]}
Outcome: Notification sent: Rejected

Test 4: Moderate scores
Submission saved: {'title': 'Research Output', 'scores': [3, 3, 4]}
Outcome: Notification sent: Revision

Test 5: High scores
Submission saved: {'title': 'Research Output', 'scores': [5, 5, 4]}
Outcome: Notification sent: Accepted
```

Figure 2: Test cases output

Test Case	Input	Outcome
Invalid submission	Missing title	Rejected: Invalid format
No reviewers	force_no_reviewers	Rejected: No reviewers available
Low scores	[1,2,2]	Rejected
Moderate scores	[3,3,4]	Revision
High scores	[5,5,4]	Accepted

Table 1: Optimised implementation results

Mapping to Decision Table

Each outcome corresponds directly to the decision table rules:

- R1: High average with consensus → Accepted
- R2: Moderate average with consensus → Revision
- R3: Low average with consensus → Rejected
- R4: No consensus → Revision
- R5: No reviewers → Rejected

The optimised system achieves cleaner separation of concerns, aligns with the improved sequence diagram, and remains functionally equivalent to the baseline. Traceability between diagrams, code, and test evidence demonstrates compliance with assignment requirements.

Click to access the COS-730 Assignment 2 Repository

Task 6: Empirical Evaluation and Comparison

A global counter is introduced to track the number of method calls executed during each workflow. The counter increments at the start of every method in the core system classes (SubmissionController, Validator, Database, ReviewerManager, EvaluationManager, NotificationService). Actors such as UI and Researcher are excluded, as they represent external entry points rather than internal processing.
After each test case, the total number of method calls was printed and reset. The following results were obtained:

Test Case	Baseline interactions	Optimised interactions
Invalid submission	12	3
No reviewers	12	7
Low scores	12	7
Moderate scores	12	7
High scores	12	7

Table 2: Comparison of interactions between Baseline and Optimised designs

The results show that the optimised design consistently reduces the number of method calls compared to the baseline. In particular, invalid submissions are rejected earlier in the workflow, **3 calls vs. 12**, demonstrating improved efficiency through streamlined validation and reduced redundant interactions. For valid submissions, the optimised design stabilises at 7 calls, whereas the baseline requires 12 calls regardless of outcome. This confirms that the optimisation achieves functional equivalence with fewer internal interactions.

Execution time benchmarking

Task 6 screenshots

Task 6 screenshots of execution runs and method calls are stored in the **Original/diagrams/Task6_Original-MethodCall&ExecutionTimes/** subfolder and the **Optimised/diagrams/Task6_Optimised-ExecutionTimes/** subfolder for traceability.

Execution time is measured using `time.perf_counter()` over ten repeated runs per test case. The baseline consistently requires 12 method calls, with execution times ranging between 0.41 ms and 1.45 ms, clustering around 0.6–1.0 ms. This

gives a stable average of approximately 0.9 ms. The optimised design reduces both calls and execution time, with averages between 0.01 ms and 0.06 ms depending on the test case.

Test Case	Baseline Calls	Baseline Avg (ms)	Optimised Calls	Optimised Avg (ms)
Invalid submission	12	0.9	3	0.01
No reviewers	12	0.9	7	0.06
Low scores	12	0.9	7	0.05
Moderate scores	12	0.9	7	0.05
High scores	12	0.9	7	0.04

Table 3: Baseline vs Optimised execution times and method calls

The optimised design executes faster due to reduced method calls and streamlined decision logic. Invalid submissions are rejected earlier, while valid submissions stabilise at lower execution times compared to the baseline. This demonstrates that optimisation improves performance without sacrificing functional correctness.

Code Complexity

The baseline design centralises responsibilities in the `SubmissionController`, which results in longer methods and higher coupling. The controller directly manages validation, database access, reviewer assignment, evaluation, and notifications. This design increases method size and reduces cohesion.
The optimised design distributes responsibilities across specialised classes. `ReviewerManager` encapsulates reviewer logic, `EvaluationManager` handles scoring rules, and `NotificationService` manages outcomes. Decision logic is externalised into a decision table, which shortens methods and improves modularity. Table 4 summarises the differences.

Aspect	Baseline	Optimised
Controller coupling	High (single orchestrator)	Reduced (distributed responsibilities)
Method length	Longer (multi-step orchestration)	Shorter (modular methods)
Decision logic	Embedded in controller	Externalised via decision table
Maintainability	Low (hard to extend)	Higher (easier to extend rules)

Table 4: Code complexity comparison between Baseline and Optimised design

The optimised design improves clarity and maintainability by reducing coupling and externalising decision logic. While it introduces additional modules, this trade-off supports long-term extensibility and reduces the risk of errors when evolving the system.

Maintainability indicators

Maintainability is evaluated using both quantitative and qualitative indicators. Quantitatively, the optimised design reduces method calls **(12 vs. 3–7)** and execution time **(0.9 ms vs. 0.01–0.06 ms)**. Methods are shorter, and responsibilities are more cohesive, which lowers complexity.

Qualitatively, the optimised design improves separation of concerns. Each class focuses on a single responsibility, which simplifies understanding and modification. Decision logic is externalised, which allows new rules to be added without altering the controller. This supports adaptability and system evolution.

The trade-off is the introduction of additional modules and abstraction layers. However, this increases clarity and reduces the likelihood of regression errors. Overall, the optimised design achieves functional equivalence with measurable efficiency gains and stronger maintainability.

References

[1] P. C. Clements, "Software architecture in practice," *Diss. Software Engineering Institute*, 2002.

[2] D. C. Kung, *Software Engineering*, Second Edition. McGraw-Hill College, 2023, ISBN: 978-1260792683.

[3] I. Sommerville, *Software Engineering*, Tenth Edition. Pearson, 2016, ISBN: 978-1-292-09613-1.

[4] M. Fowler, *Patterns of enterprise application architecture*. Addison-Wesley, 2012.

[5] GeeksforGeeks. "Software engineering — introduction." [Online; accessed April 27, 2026]. [Online]. Available: https://www.geeksforgeeks.org/software-engineering-introduction/.